



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Data Science in Online Marketing

Recommenders – Collaborative Filtering

Maarten Soomer



Course Outline

Date	Topic	Assignment
3 april	Recommenders: Content Based	
10 april	Recommenders: Collaborative Filtering	Yes
17 april	Recommenders: Guest Lecture Bol.com Online advertising: Introduction	
24 april	Online advertising: Reinforcement Learning	
1 may	Online advertising: CTR prediction	Yes
15 may	Online advertising: Media Mix Modeling & Conversion Attribution	
22 may	Revision	
29 may	Digital exam on location	

Recommendations



NETFLIX

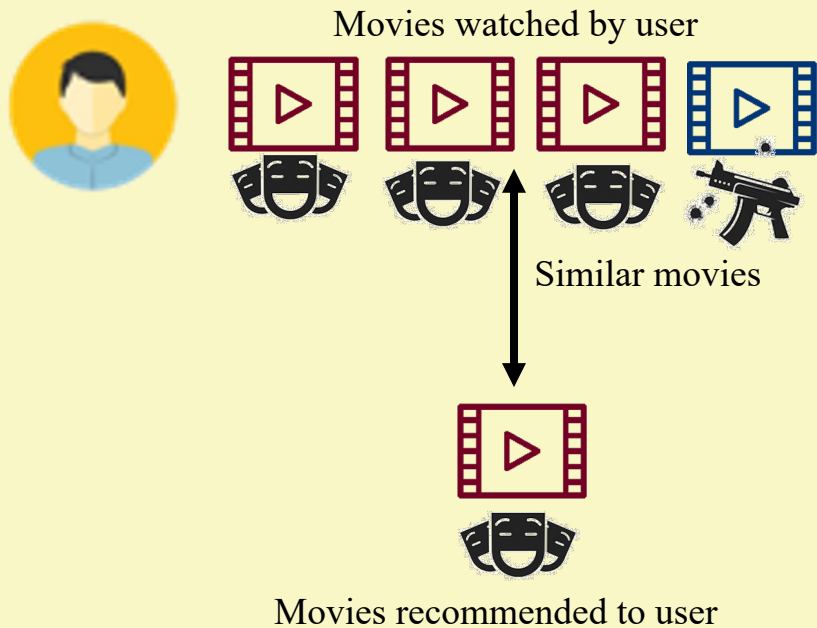


Customers Who Bought This Item Also Bought

Book Title	Author	Rating	Format	Price	Prime
Understanding Social Networks: Theories, ...	Charles Kadushin	★★★★★ (8)	Paperback	\$16.09	✓ Prime
Social Network Analysis: Methods and ...	Stanley Wasserman	★★★★★ (14)	Paperback	\$47.81	✓ Prime
Social Network Analysis: History, Theory and ...	Christina Prell	★★★★★ (7)	Paperback	\$42.40	✓ Prime
Social Network Analysis	John Scott		Paperback	\$39.48	✓ Prime
Exponential Random Models for Social ...	Dean Lusher	★★★★★ (4)	Paperback	\$31.49	✓ Prime

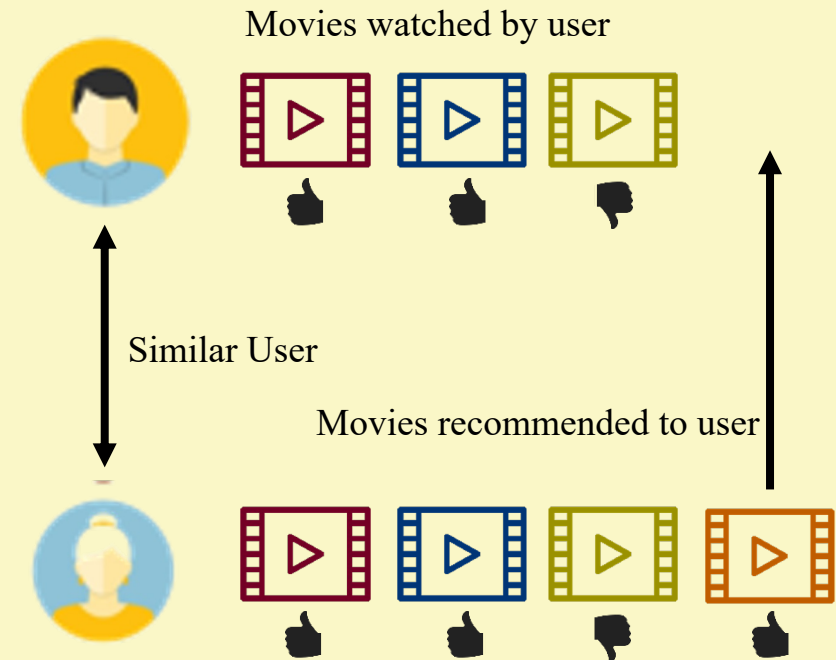
Techniques

Content-based



Similarity based on **items characteristics**: e.g., genre

Collaborative Filtering



Similarity based on **users taste** (feedback)



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Collaborative Filtering



Collaborative Filtering

General idea:

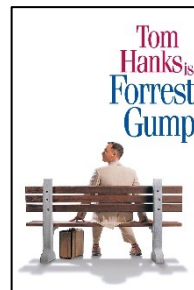
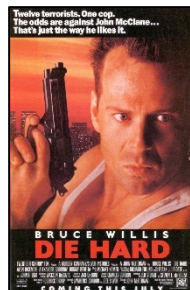
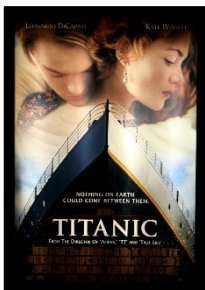
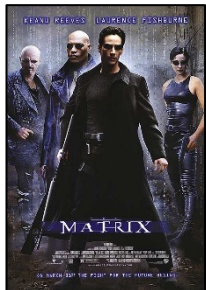
- Recommend items based on ratings from other users
 - ☐ items that **similar users** like *OR*
 - ☐ items that are **similar** to **items** that you like

Required:

- Past user feedback
- Past feedback of other users

User Feedback - Rating Matrix

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
Alice	5	3	4	4	?
John	3	1	2	3	3
Lucy	4	3	4	3	5
Eric	3	3	1	5	4
Diane	3	5	5	3	1



Rating Matrix - Notation

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
Alice	5	3	4	4	?
John	3	1	2	3	3
Lucy	4	3	4	3	5
Eric	3	3	1	5	4
Diane	3	5	5	3	1

- $R = n \times m$ matrix with $r_{u,i}$ = rating of user u for item i
- n users
- m items



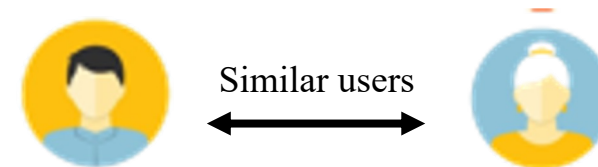
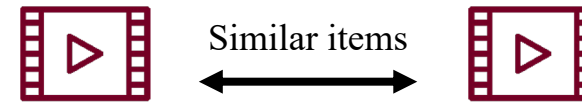
Collaborative Filtering Models

- User-Based Nearest Neighborhood
- Item-Based Nearest Neighborhood
- Matrix Factorization

Similarity Measures

How to measure similarity between users or items?

- Dice Coefficient
- (Adjusted) Cosine similarity
- **Pearson's correlation coefficient**





UNIVERSITY OF AMSTERDAM
Amsterdam Business School

User-Based Nearest Neighborhood

User-Based Nearest Neighborhood

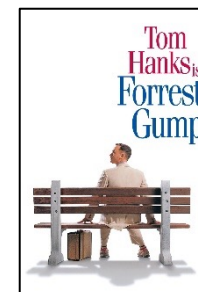
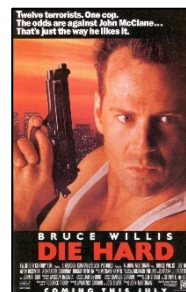
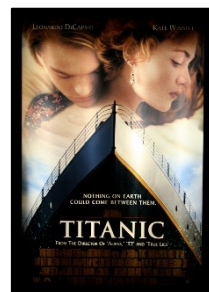
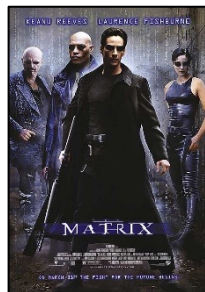
1. Determine user similarities based on ratings
2. Predict a rating for each unseen item based on ratings of similar users
3. Recommend the item with the highest predicted rating

User-Based Nearest Neighborhood

- Based on the rating matrix, calculate the similarities between each pair of users
- For each user u
 - For each unrated item i from user u
 - Construct the set $N_{u,i}$ with the k users that rated item i , that are most similar to u
 - Predict the rating of item i of user u as the average rating over all users in the set $N_{u,i}$
 - Recommend the item i^* to user u that has the highest predicted rating

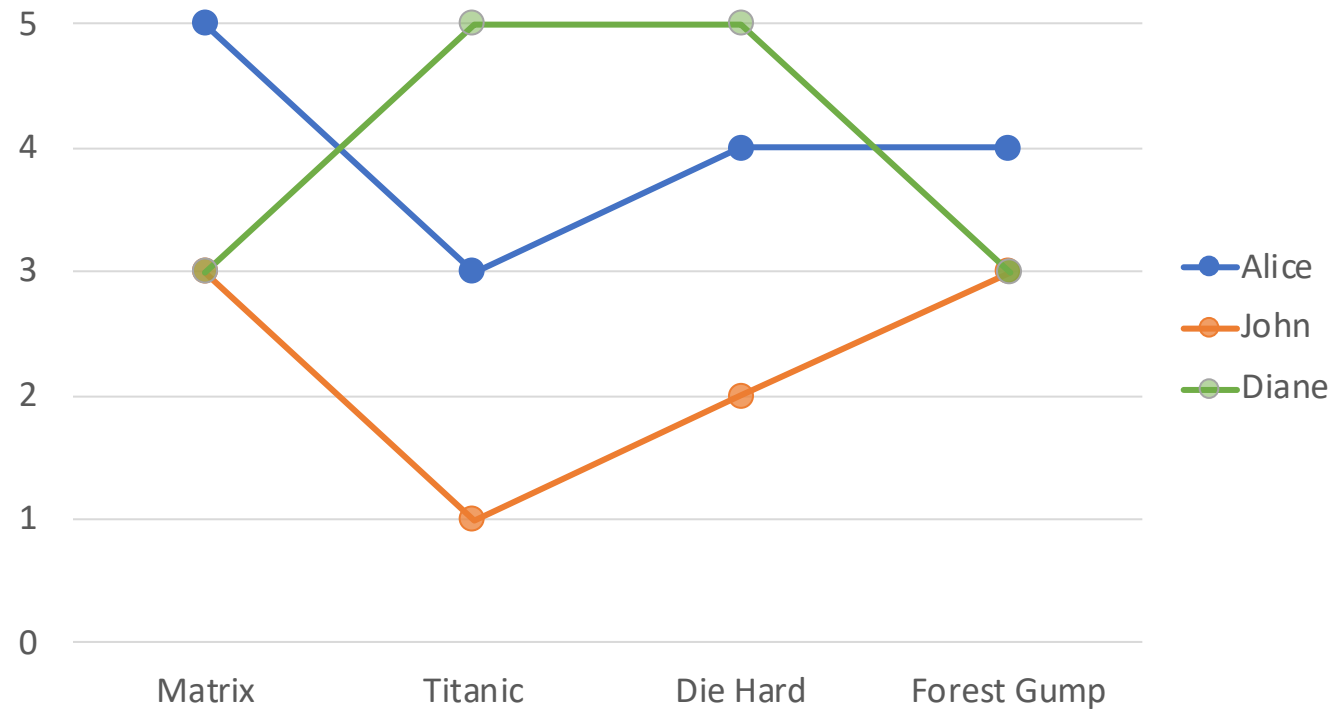
Example: predict Alice's rating for Wall-E

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
Alice	5	3	4	4	?
John	3	1	2	3	3
Lucy	4	3	4	3	5
Eric	3	3	1	5	4
Diane	3	5	5	3	1



Similarity Measure

Are John's or Diane's ratings more similar to Alice's?



Similarity Measure

- Many different measures for similarities between users
- Most common measure is Pearson's correlation coefficient:

$$\text{sim}(u, v) = \frac{\sum_{i \in I} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{i \in I} (r_{v,i} - \bar{r}_v)^2}}$$

With:

- $\text{sim}(u, v)$ similarity between users u and v
- I set of items rated by both users u and v
- $r_{u,i}$ rating of user u for item i
- $\bar{r}_u$ average rating of user u over all rated items

Pearson's correlation coefficient explained

Some users are very generous raters, other very conservative raters. That doesn't have to mean their taste is different.

To account for this look at correlation between normalised ratings:

$$\frac{(r_{u,i} - \bar{r}_u)}{\sqrt{\sum_{i \in I} (r_{u,i} - \bar{r}_u)^2}}$$

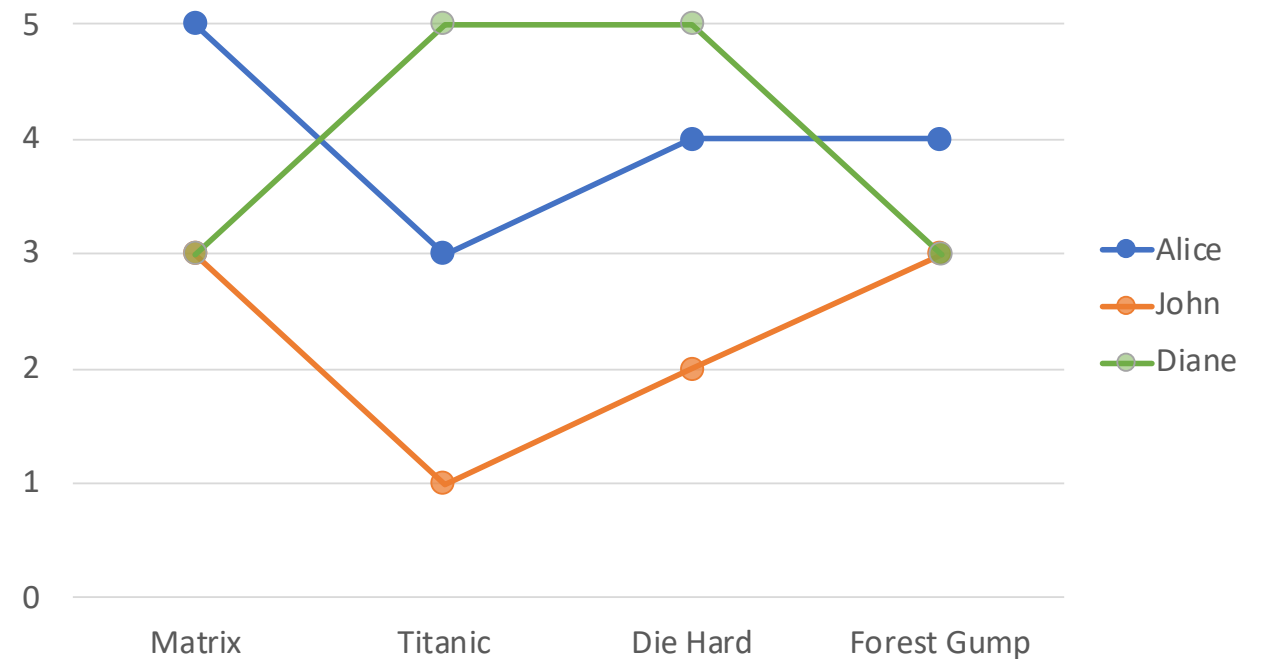
- $r_{u,i}$ rating of user u for item i
- $\bar{r}_u$ average rating of user u over all rated items

Normalised ratings look at deviation from user's average rating compared to total variance in the user's ratings

Example - Pearson's correlation coefficient

$$\text{sim}(u, v) = \frac{\sum_{i \in I} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{i \in I} (r_{v,i} - \bar{r}_v)^2}}$$

- $\text{sim}(\text{Alice}, \text{John}) = 0.84$
- $\text{sim}(\text{Alice}, \text{Diane}) = -0.61$



Alice and John have similar tastes, but their absolute ratings are different

Example - Pearson's correlation coefficient

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$	$\sqrt{\sum_{i \in I} (r_{u,i} - \bar{r}_u)^2}$
Alice	5	3	4	4	?	4.0	1.4
John	3	1	2	3	3	2.4	1.7
Lucy	4	3	4	3	5	3.8	1.2
Eric	3	3	1	5	4	3.2	2.9
Diane	3	5	5	3	1	3.4	2.3

Correlation between Alice and John:

- Set of items rated by both Alice and John: $I = \{\text{Matrix, Titanic, Die Hard, Forrest Gump}\}$
- $$\text{sim}(\text{Alice, John}) = \frac{(5-4)*(3-2.4) + (3-4)*(1-2.4) + (4-4)*(2-2.4) + (4-4)*(3-2.4)}{1.4*1.7} = 0.84$$

Example - Pearson's correlation coefficient

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$	$\sqrt{\sum_{i \in I} (r_{u,i} - \bar{r}_u)^2}$
Alice	5	3	4	4	?	4.0	1.4
John	3	1	2	3	3	2.4	1.7
Lucy	4	3	4	3	5	3.8	1.2
Eric	3	3	1	5	4	3.2	2.9
Diane	3	5	5	3	1	3.4	2.3

- $sim(Alice, John) = 0.84$

- $sim(Alice, Lucy) = 0.61$

- $sim(Alice, Eric) = 0.00$

- $sim(Alice, Diane) = -0.61$

2 Nearest Neighbours are John and Lucy

$N_{Alice,i} = \{John, Lucy\}$ for each item i

Predicted Rating

- Most common prediction calculation:

$$pred(u, i) = \bar{r}_u + \frac{\sum_{v \in N_{u,i}} sim(u, v) * (r_{v,i} - \bar{r}_v)}{\sum_{b \in N_{u,p}} sim(u, v)}$$

With:

- $pred(u, i)$ predicted rating for user u for item i
- $N_{u,i}$ set of most similar users (nearest neighbors) of user u that rated item i
- $r_{u,i}$ rating of user u for item i
- $\bar{r}_u$ average rating of user u over all rated items

Example: predict Alice's rating for Wall-E

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$
Alice	5	3	4	4	?	4.0
John	3	1	2	3	3	2.4
Lucy	4	3	4	3	5	3.8

2 Nearest Neighbours are John and Lucy: $N_{Alice} = \{\text{John, Lucy}\}$

- $sim(\text{Alice, John}) = 0.84$
- $sim(\text{Alice, Lucy}) = 0.61$

$Pred(\text{Alice, Wall-E})$

Example: predict Alice's rating for Wall-E

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$
Alice	5	3	4	4	?	4.0
John	3	1	2	3	3	2.4
Lucy	4	3	4	3	5	3.8

2 Nearest Neighbours are John and Lucy: $N_{Alice} = \{\text{John, Lucy}\}$

- $sim(\text{Alice, John}) = 0.84$
- $sim(\text{Alice, Lucy}) = 0.61$

$$Pred(\text{Alice, Wall-E}) = 4 + \frac{0.84 * (3 - 2.4) + 0.61 * (5 - 3.8)}{0.84 + 0.61} = 4.85$$

About User-Based kNN

Pros:

- User preferences to capture qualitative features such as taste
- Less receptive to offering “more of the same” (serendipity)
- No data required on item characteristics
- Good at capturing local effects

Cons:

- Not useful for new users and new items (cold start)
- Unreliable results if there are few user ratings (sparsity)
- High computation time for large number of users (scalability)



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Item-Based Nearest Neighborhood



Item-Based Nearest Neighborhood

1. Determine item similarities based on ratings
2. Predict a rating for each unseen item based on ratings of similar rated items
3. Recommend the item with the highest predicted rating

Item-Based Nearest Neighborhood

- Based on the rating matrix, calculate the similarities between each pair of items
- For each user u
 - For each unrated item i from user u
 - Construct the set $N_{u,i}$ with the k items rated by user u , that are most similar to item i
 - Predict the rating of item i of user u as the average rating over the items in the set $N_{u,i}$
 - Recommend the item i^* to user u that has the highest predicted rating

Similarity Measure

- Many different measures for similarities between items
- Most common measure is adjusted cosine measure:

$$sim(i, j) = \frac{\sum_{u \in U} (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,j} - \bar{r}_u)^2}}$$

With:

- $sim(i, j)$ similarity between items i and j
- U set of all users that have rated both items i and j
- $r_{u,i}$ rating of user u for item i
- $\bar{r}_u$ average rating of user u over all rated items

Example - adjusted cosine measure

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$
Alice	5	3	4	4	?	4.0
John	3	1	2	3	3	2.4
Lucy	4	3	4	3	5	3.8
Eric	3	3	1	5	4	3.2
Diane	3	5	5	3	1	3.4
$\sqrt{\sum_{u \in U} (r_{i,u} - \bar{r}_u)^2}$	0.8	2.3	2.8	2.1	2.9	

Similarity between Matrix and Wall-E:

- $U = \{\text{John, Lucy, Eric, Diane}\}$
- $\text{sim}(\text{Matrix, Wall-E}) =$

$$\text{sim}(i, j) = \frac{\sum_{u \in U} (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,j} - \bar{r}_u)^2}}$$

Example - adjusted cosine measure

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$
Alice	5	3	4	4	?	4.0
John	3	1	2	3	3	2.4
Lucy	4	3	4	3	5	3.8
Eric	3	3	1	5	4	3.2
Diane	3	5	5	3	1	3.4
$\sqrt{\sum_{u \in U} (r_{u,i} - \bar{r}_u)^2}$	0.8	2.3	2.8	2.1	2.9	

Similarity between Matrix and Wall-E:

- $U = \{\text{John, Lucy, Eric, Diane}\}$

- $\text{sim}(\text{Matrix, Wall-E}) =$

$$\frac{(3-2.4)*(3-2.4) + (4-3.8)*(5-3.8) + (3-3.2)*(4-3.2) + (3-3.4)*(1-3.4)}{0.8*2.9} = 0.63$$

$$\text{sim}(i, j) = \frac{\sum_{u \in U} (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,j} - \bar{r}_u)^2}}$$

Example - adjusted cosine measure

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E	$\bar{r}_u$
Alice	5	3	4	4	?	4.0
John	3	1	2	3	3	2.4
Lucy	4	3	4	3	5	3.8
Eric	3	3	1	5	4	3.2
Diane	3	5	5	3	1	3.4
$\sqrt{\sum_{u \in U} (r_{u,i} - \bar{r}_u)^2}$	0.8	2.3	2.8	2.1	2.9	

■ $sim(\text{Matrix}, \text{Wall-E}) = 0.63$

■ $sim(\text{Titanic}, \text{Wall-E}) = -0.89$

■ $sim(\text{Die Hard}, \text{Wall-E}) = -0.71$

■ $sim(\text{Forrest Gump}, \text{Wall-E}) = 0.30$

2 Nearest Neighbours: Matrix and Forrest Gump

$$N_{alice, Wall-E} = \{\text{Matrix}, \text{Forrest Gump}\}$$

Note that...

- Pearson's correlation coefficient calculates the similarity between users
 - Users are similar if their (normalised) ratings behave the same over the items
- Adjusted cosine measure calculates the similarity between items
 - Items are similar if their (normalised) ratings behave the same over the users
- If we predict the rating based on other user's ratings, we need to correct for user bias (e.g. Alice is more optimistic than John)
- If we predict the rating based on other item's ratings (**from the same user**) we don't need to correct for user bias

Predicted Rating

■ Most common prediction technique is similarity-weighted average:

$$pred(u, i) = \frac{\sum_{j \in N_{u,i}} sim(i, j) * r_{u,j}}{\sum_{j \in N_{u,i}} sim(i, j)}$$

With:

$pred(u, i)$ predicted rating for user u for item i

$N_{u,i}$ set of most similar items (nearest neighbors) of item i that have been rated by user u

$r_{u,j}$ rating of user u for item j

Example: predict Alice's rating for Wall-E

	Matrix	Forrest Gump	Wall-E
Alice	5	4	?

- 2 Nearest Neighbours: Matrix and Forest Gump:

$$N_{alice, Wall-E} = \{\text{Matrix, Forrest Gump}\}$$

- $sim(\text{Matrix, Wall-E}) = 0.63$
- $sim(\text{Forrest Gump, Wall-E}) = 0.30$

$$Pred(\text{Alice, Wall-E}) =$$

Example: predict Alice's rating for Wall-E

	Matrix	Forrest Gump	Wall-E
Alice	5	4	?

- 2 Nearest Neighbours: Matrix and Forest Gump:

$$N_{alice, Wall-E} = \{\text{Matrix, Forrest Gump}\}$$

- $sim(\text{Matrix, Wall-E}) = 0.63$
- $sim(\text{Forrest Gump, Wall-E}) = 0.30$

$$Pred(\text{Alice, Wall-E}) = \frac{0.63 * 5 + 0.30 * 4}{0.63 + 0.30} = 4.68$$

About Item-Based kNN

Pros:

- User preferences to capture qualitative features such as taste
- Less receptive to offering “more of the same” (serendipity)
- No data required on item characteristics
- Good at capturing local effects

Cons:

- Not useful for new users and new items (cold start)
- Unreliable results if there are few user ratings (sparsity)
- High computation time for large number of items (scalability)



Extensions

Variance weighting

- Some items are universally liked/disliked
- Not very informative that users A and B both like “The Godfather”
- Give higher weight for items with a higher variance

Significance weighting

- Some users have co-rated only a few items
- They can agree by chance
- Give higher weight for users that have more items in common

Overview kNN Methods

Content-Based (previous week)

- Predicted rating = average rating of similar items
- Item similarity based on the item characteristics (e.g., genre, writer, actor)

User-Based

- Predicted rating = average rating of similar users
- User similarity based on how they rate the same item

Item-Based

- Predicted rating = average rating of similar items
- Item similarity based on how they are rated by the same user



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Matrix Factorization

Sparsity of Rating Matrix

- How many of the 10,000+ Netflix titles did you watch?
- How many of the 30+ million Spotify songs did you listen to?

The ratings matrix is usually highly incomplete (sparse)

- Users and items have very few common ratings

		<i>Items</i>					
		<i>1</i>	<i>2</i>	...	<i>i</i>	...	<i>m</i>
<i>Users</i>	<i>1</i>	5	3		1	2	
	<i>2</i>		2				4
	:			5			
	<i>u</i>	3	4		2	1	
	:					4	
	<i>n</i>			3	2		
		<i>a</i>	3	5		?	1

Sparsity of Rating Matrix

- Collaborative Filtering has the very strong advantage that it uses user behaviour to capture qualitative features such as taste and quality
- But the nearest neighbourhood methods suffer from a sparse rating matrix:
 - For many users we don't observe that they are similar because they don't have any ratings in common
 - The similarities we do observe are unreliable because of limited observations

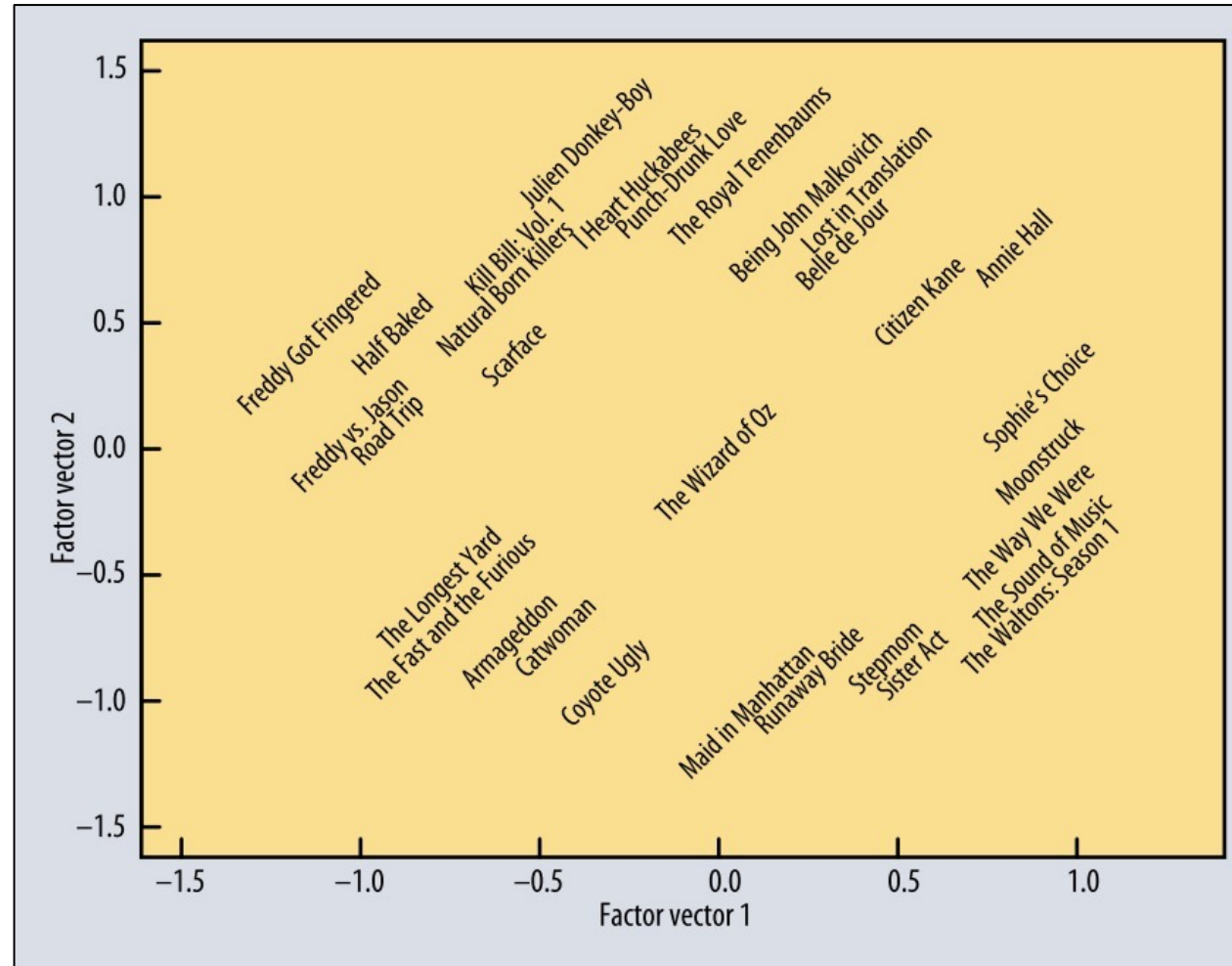
Matrix Factorization

- A Collaborative Filtering technique that aims to uncover latent factors that explain the observed ratings matrix
- It captures as much information as possible from the rating matrix into a limited number of latent factors

Example:

- For 1,000 users and 1,000 items we have $1,000 * 1,000 = 1,000,000$ entries in the ratings matrix (of which most will be missing)
- If we introduce 5 latent factors, then we have $1,000 * 5 = 5,000$ user scores and $1,000 * 5 = 5,000$ item scores

Example - Top 2 Latent Factors Netflix





Latent Factors

- Introduce L latent factors (unobserved)
- “Learn” user and item latent factor scores based on patterns in the ratings data
- Always a surprise what latent factors the data provides you: no inherent meaning

Notation

- r_{ui} = rating of user u for item i
- q_{il} = score of item i on latent factor l (item characteristics)
- p_{ul} = score of user u on latent factor l (user preferences)

$$q_i = \begin{pmatrix} q_{i1} \\ \dots \\ q_{iL} \end{pmatrix} \quad p_u = \begin{pmatrix} p_{u1} \\ \dots \\ p_{uL} \end{pmatrix}$$

Note that we don't necessarily mean observed item characteristics such as genre or writer.

But also unobserved item characteristics such as taste or quality.

- Predicted rating: $\hat{r}_{ui} = q_i^T p_u$



Matrix Factorization

- Predicted rating: $\hat{r}_{ui} = q_i^T p_u = q_{i1} * p_{u1} + q_{i2} * p_{u2} + \dots + q_{iL} * p_{uL}$

- Find p and q that minimize SSE:

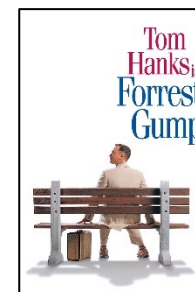
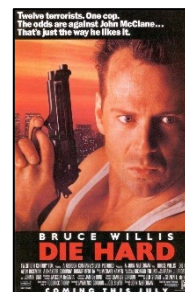
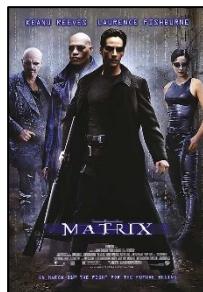
$$\min_{q,p} \sum_{(u,i) \in \kappa} (r_{ui} - \hat{r}_{ui})^2$$

where:

- κ = set of known ratings

Example: predict Alice's rating for Wall-E

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
Alice	5	3	4	4	?
John	3	1	2	3	3
Lucy	4	3	4	3	5
Eric	3	3	1	5	4
Diane	3	5	5	3	1



	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
Alice	5	3	4	4	?
John	3	1	2	3	3
Lucy	4	3	4	3	5
Eric	3	3	1	5	4
Diane	3	5	5	3	1

$\approx$

	1	2
Alice	$p_{Alice,1}$	$p_{Alice,2}$
John	$p_{John,1}$	$p_{John,2}$
Lucy	$p_{Lucy,1}$	$p_{Lucy,2}$
Eric	$p_{Eric,1}$	$p_{Eric,2}$
Diane	$p_{Diane,1}$	$p_{Diane,2}$

*

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
1	$q_{Matrix,1}$	$q_{Titanic,1}$	$q_{DieHard,1}$	$q_{FG,1}$	$q_{WallE,1}$
2	$q_{Matrix,2}$	$q_{Titanic,2}$	$q_{DieHard,2}$	$q_{FG,2}$	$q_{WallE,2}$

Example: predict Alice's rating for Wall-E

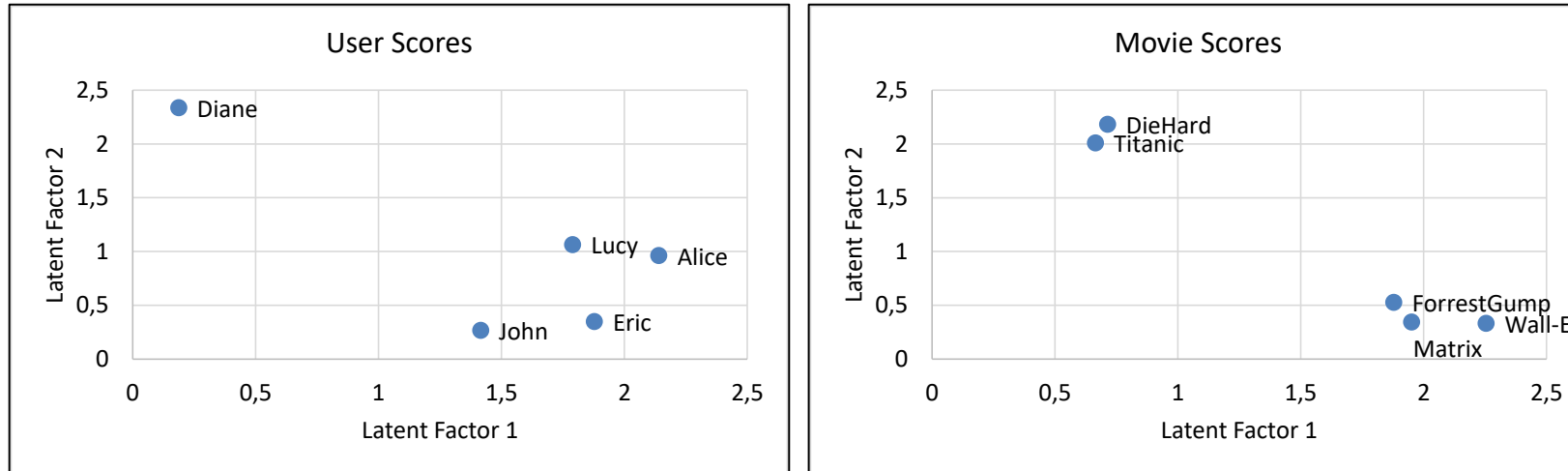
$$Pred(\text{Alice}, \text{Wall-E}) = p_{\text{Alice},1} * q_{\text{WallE},1} + p_{\text{Alice},2} * q_{\text{WallE},2}$$

	1	2
Alice	$p_{\text{Alice},1}$	$p_{\text{Alice},2}$
John	$p_{\text{John},1}$	$p_{\text{John},2}$
Lucy	$p_{\text{Lucy},1}$	$p_{\text{Lucy},2}$
Eric	$p_{\text{Eric},1}$	$p_{\text{Eric},2}$
Diane	$p_{\text{Diane},1}$	$p_{\text{Diane},2}$

*

	Matrix	Titanic	Die Hard	Forrest Gump	Wall-E
1	$q_{\text{Matrix},1}$	$q_{\text{Titanic},1}$	$q_{\text{DieHard},1}$	$q_{\text{FG},1}$	$q_{\text{WallE},1}$
2	$q_{\text{Matrix},2}$	$q_{\text{Titanic},2}$	$q_{\text{DieHard},2}$	$q_{\text{FG},2}$	$q_{\text{WallE},2}$

Example



- We capture similarities between users and movies in 2 latent factors
- In total we estimate 20 parameters (2 factors for 5 movies and 5 users)
- Diane rates very different from the other users
- Die Hard and Titanic are rated similar but different from the other movies

Predicted rating based on latent factors

- Directly based on latent factor scores

$$\text{pred}(\text{Alice}, \text{Wall-E}) = q_{\text{Wall-E}}^T p_{\text{Alice}} = \begin{pmatrix} 2.26 \\ 0.33 \end{pmatrix}^T \begin{pmatrix} 2.14 \\ 0.96 \end{pmatrix} = 5.15$$

- Average over k nearest users of Alice based on latent factor scores

$$\text{pred}(\text{Alice}, \text{Wall-E}) = \frac{r_{\text{Lucy}, \text{Wall-E}} + r_{\text{Eric}, \text{Wall-E}}}{2} = \frac{5+4}{2} = 4.5 \quad (k=2)$$

- Average over k nearest movies of Wall-E based on latent factor scores

$$\text{pred}(\text{Alice}, \text{Wall-E}) = \frac{r_{\text{Alice}, \text{Matrix}} + r_{\text{Alice}, \text{ForrestGump}}}{2} = \frac{5+4}{2} = 4.5 \quad (k=2)$$

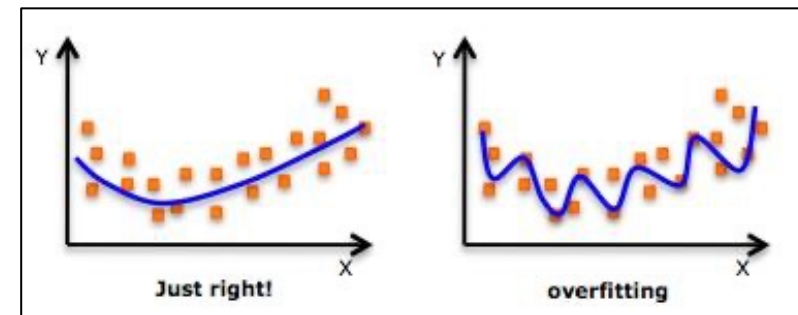
Overfitting

- With 1,000 users and 1,000 items we have 1,000,000 entries in the rating matrix, but maybe only 8,000 ratings that are actually present
- If we introduce 5 latent factors, then we have 10,000 parameters $\rightarrow$ more parameters than observations
- Let's penalize the magnitude of p and q to prevent overfitting:

$$\min_{q,p} \sum_{(u,i) \in \kappa} (r_{ui} - q_i^T p_u)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

where:

- κ = set of known ratings
- λ = hyperparameter



But there is a catch...

- How to find the p and q values that minimize the error?
- We typically have:
 - ❑ Tens/Hundreds of thousands of items
 - ❑ Tens/Hundreds of thousands of users
- Common least squares or Max Likelihood estimators can not handle such a big rating matrix
- We also want to easily update our parameters for each new rating
 - **Stochastic Gradient Descent**

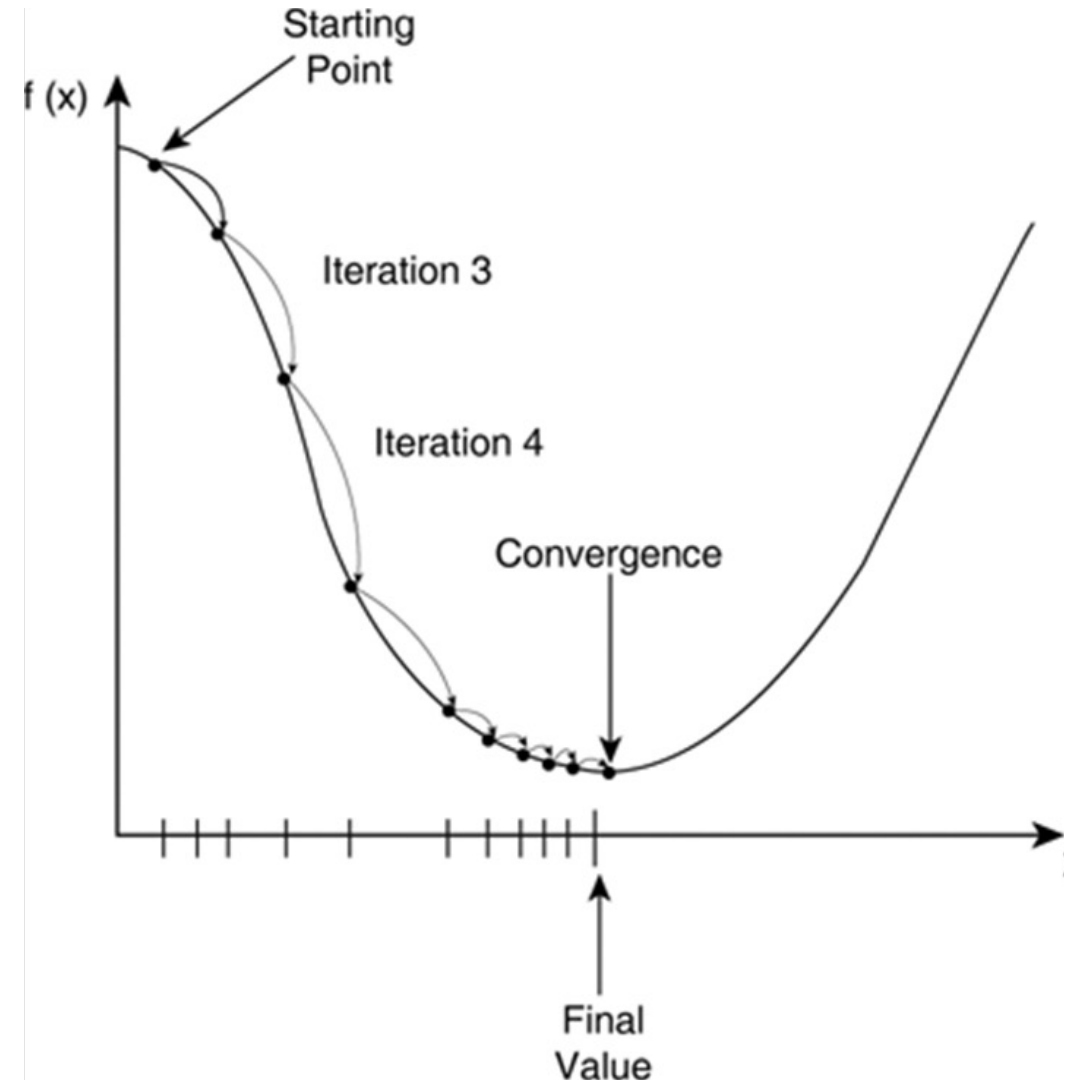


UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Matrix Factorization using Stochastic Gradient Descent

Stochastic Gradient Descent

- Online learning algorithm: can learn one observation (rating) at a time.
- Very efficient to perform updates, also when the number of users and items is very large.



Stochastic Gradient Descent

Minimize the loss function (error):

$$\min_{q,p} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - q_i^T p_u)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

Update the latent factor scores every time a new rating r_{ui} comes in:

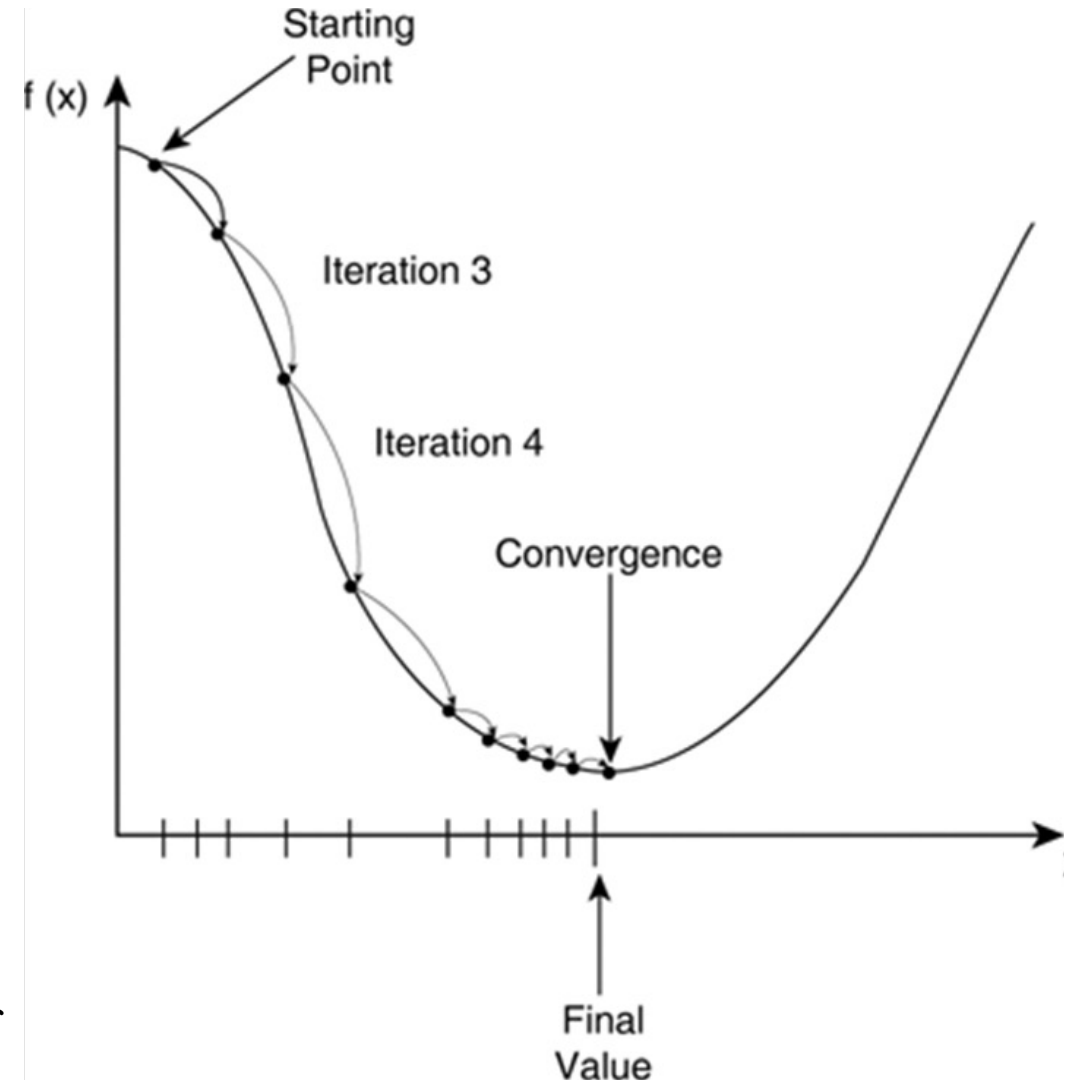
- Predict rating based on current latent factor scores: $q_i^T p_u$
- Adjust the latent factor scores to lower the error for this rating $(r_{ui} - q_i^T p_u)$:

$$q_i := q_i + \eta_t [(r_{ui} - q_i^T p_u) p_u - \lambda q_i]$$

$$p_u := p_u + \eta_t [(r_{ui} - q_i^T p_u) q_i - \lambda p_u]$$

With

η_t : learning rate (hyperparameter) determines the magnitude of the adjustment at time t



Stochastic Gradient Descent

Update of the latent factor scores:

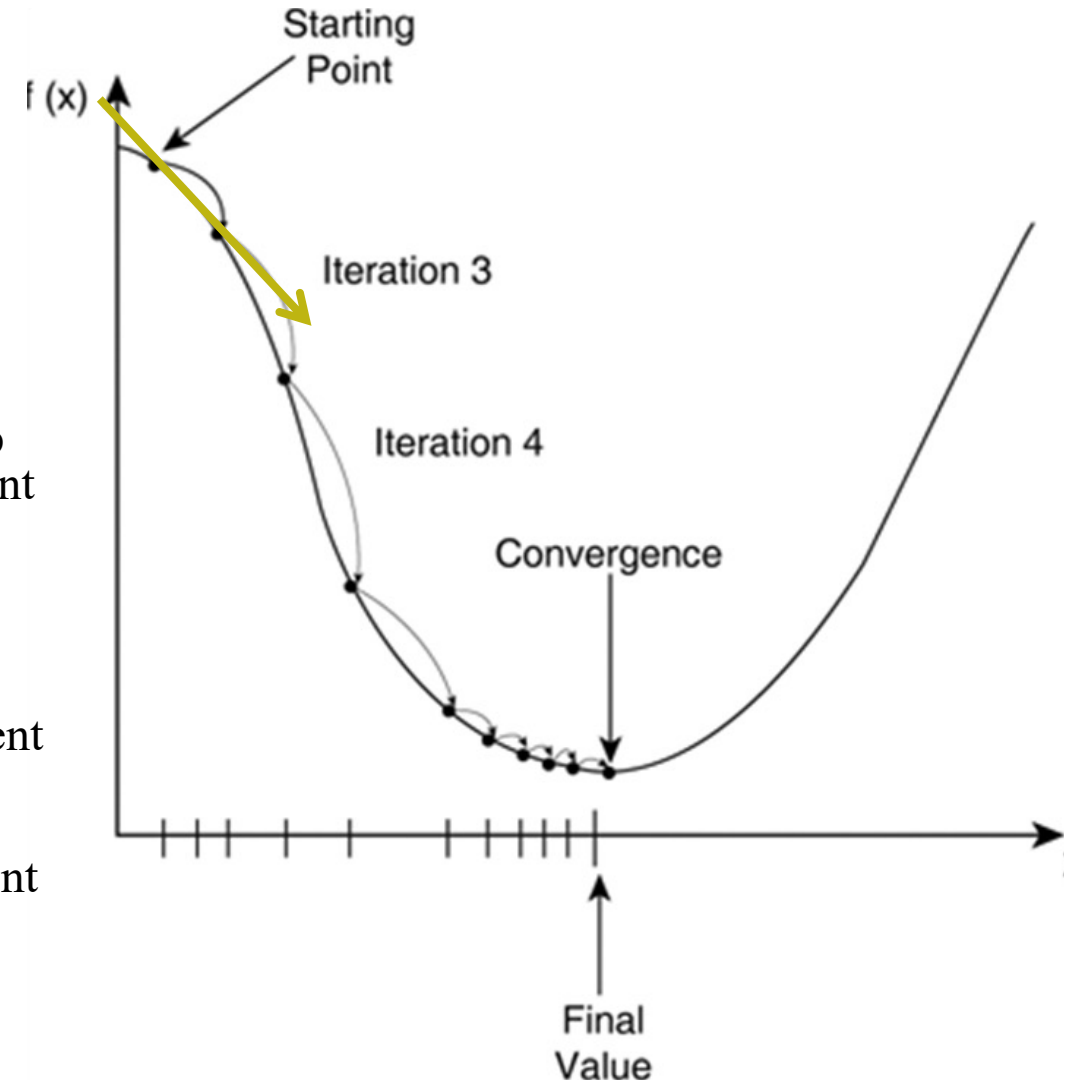
$$q_i := q_i + \eta_t [(r_{ui} - q_i^T p_u) p_u - \lambda q_i]$$

$$p_u := p_u + \eta_t [(r_{ui} - q_i^T p_u) q_i - \lambda p_u]$$

η_t is typically constant for recommenders → we don't want to stop learning, but pick up any change in preferences from the most recent ratings

The update is determined by the size of the error $(r_{ui} - q_i^T p_u)$:

- When the rating is higher than predicted we increase the latent factor scores
- When the rating is lower than predicted we decrease the latent factor scores
- No change in latent factor scores for other items and users



SGD Parameter Update - Example

- Update parameters based on observation $r_{u,i}$:
- Using ratings of Alice, John and Lucy for Matrix and Wall-E
- $L=2$, $\lambda = 0$ (no regularization), $\eta_t = 0.2$

Latent Factor Scores initialized
randomly (between 0 and 2.5)

Alice 1	Alice 2	John 1	John 2	Lucy 1	Lucy 2	Matrix 1	Matrix 2	Wall-E 1	Wall-E 2
0.1	1.1	1.6	2.0	0.4	2.4	0.4	2.2	1.0	1.2

SGD Parameter Update - Example

- Update parameters based on observation $r_{u,i}$:
- Using ratings of Alice, John and Lucy for Matrix and Wall-E
- $L=2$, $\lambda = 0$ (no regularization), $\eta_t = 0.2$

Alice 1	Alice 2	John 1	John 2	Lucy 1	Lucy 2	Matrix 1	Matrix 2	Wall-E 1	Wall-E 2	User	Movie	Rating	Prediction	Error
0.1	1.1	1.6	2.0	0.4	2.4	0.4	2.2	1.0	1.2	Alice	Matrix	5	2.6	2.4

Processing the rating of Alice for Matrix:

Prediction by adding the products of the factors :
 $0.1 \cdot 0.4 + 1.1 \cdot 2.2 = 2.6$

Prediction is lower than observed rating:
 $\text{Error} = 5 - 2.6 = 2.4$

SGD Parameter Update - Example

- Update parameters based on observation $r_{u,i}$:
- Using ratings of Alice, John and Lucy for Matrix and Wall-E
- $L=2$, $\lambda = 0$ (no regularization), $\eta_t = 0.2$

Alice 1	Alice 2	John 1	John 2	Lucy 1	Lucy 2	Matrix 1	Matrix 2	Wall-E 1	Wall-E 2	User	Movie	Rating	Prediction	Error
0.1	1.1	1.6	2.0	0.4	2.4	0.4	2.2	1.0	1.2	Alice	Matrix	5	2.6	2.4
0.3	2.2	1.6	2	0.4	2.4	0.5	2.8	1.0	1.2					

Update Latent Factor Scores for Alice
and the Matrix based on error

$$\text{Alice 1} = 0.1 + 0.2 * 2.4 * 0.4 = 0.3$$

SGD Parameter Update - Example

- Update parameters based on observation $r_{u,i}$:
- Using ratings of Alice, John and Lucy for Matrix and Wall-E
- $L=2$, $\lambda = 0$ (no regularization), $\eta_t = 0.2$

Alice 1	Alice 2	John 1	John 2	Lucy 1	Lucy 2	Matrix 1	Matrix 2	Wall-E 1	Wall-E 2	User	Movie	Rating	Prediction	Error
0.1	1.1	1.6	2.0	0.4	2.4	0.4	2.2	1.0	1.2	Alice	Matrix	5	2.6	2.4
0.3	2.2	1.6	2	0.4	2.4	0.5	2.8	1.0	1.2	John	Matrix	3	6.5	-3.5
0.3	2.2	1.3	0.1	0.4	2.4	-0.6	1.4	1.0	1.2	John	Wall-E	3	1.5	1.5
0.3	2.2	1.6	0.5	0.4	2.4	-0.6	1.4	1.4	1.3	Lucy	Wall-E	5	3.6	1.4
0.3	2.2	1.6	0.5	0.8	2.8	-0.6	1.4	1.5	1.9	Lucy	Matrix	4	-0.6	4.5
0.3	2.2	1.6	0.5	0.2	4	0.1	3.8	1.5	1.9	Alice	Wall-E		4.8	

Efficiency

- SGD is not optimal, but very efficient when rating matrix is sparse → we don't calculate with the full matrix, but only with the limited number of ratings we have
- Every time a new rating comes in, we perform one simple update of our parameters (latent factors scores) for the corresponding movie and user

About Matrix Factorization

Very popular Collaborative Filtering technique because:

- Improved speed: MF reduces calculation time because the huge ratings matrix is replaced by a limited number of user scores and item scores
- Improved reach: MF observes similarities between users even if they don't have any ratings in common
- Improved quality: The observed similarities are more reliable because they are based on all ratings
- Drawback is that MF does not easily pick up local similarities, while for example Nearest Neighborhood is very good at observing those (e.g. 2 users that behave almost exactly the same)



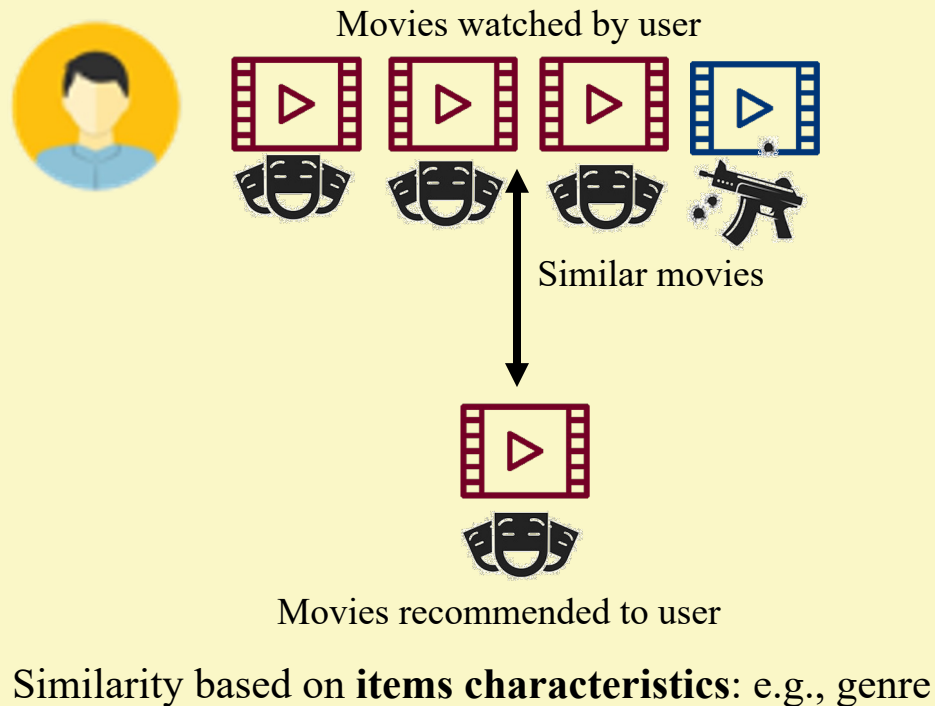
UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Hybrid Models

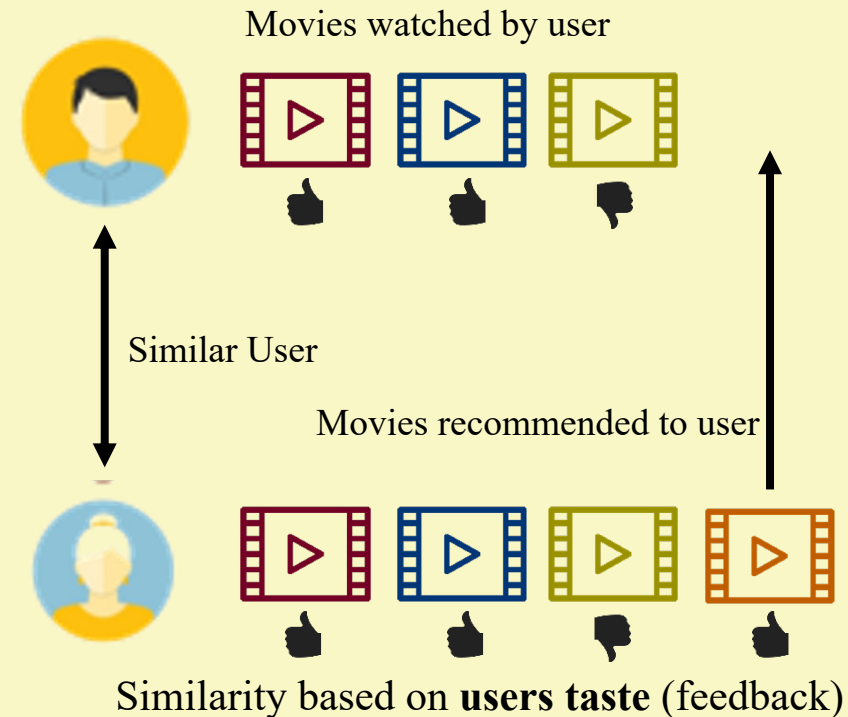
Hybrid Models



Content-based



Collaborative Filtering



Hybrid Models

Different models pick up different similarities:

- Content-Based vs. User-Based vs. Item-Based
- Local Nearest Neighbors vs. Global Latent Factors

Winners of Netflix prize:

“Our experience is that most efforts should be concentrated in deriving substantially different approaches, rather than refining a single technique”

Hybrid Models

Weighted average over model predictions

- Recommend the item(s) with the highest weighted average score over a number of different models

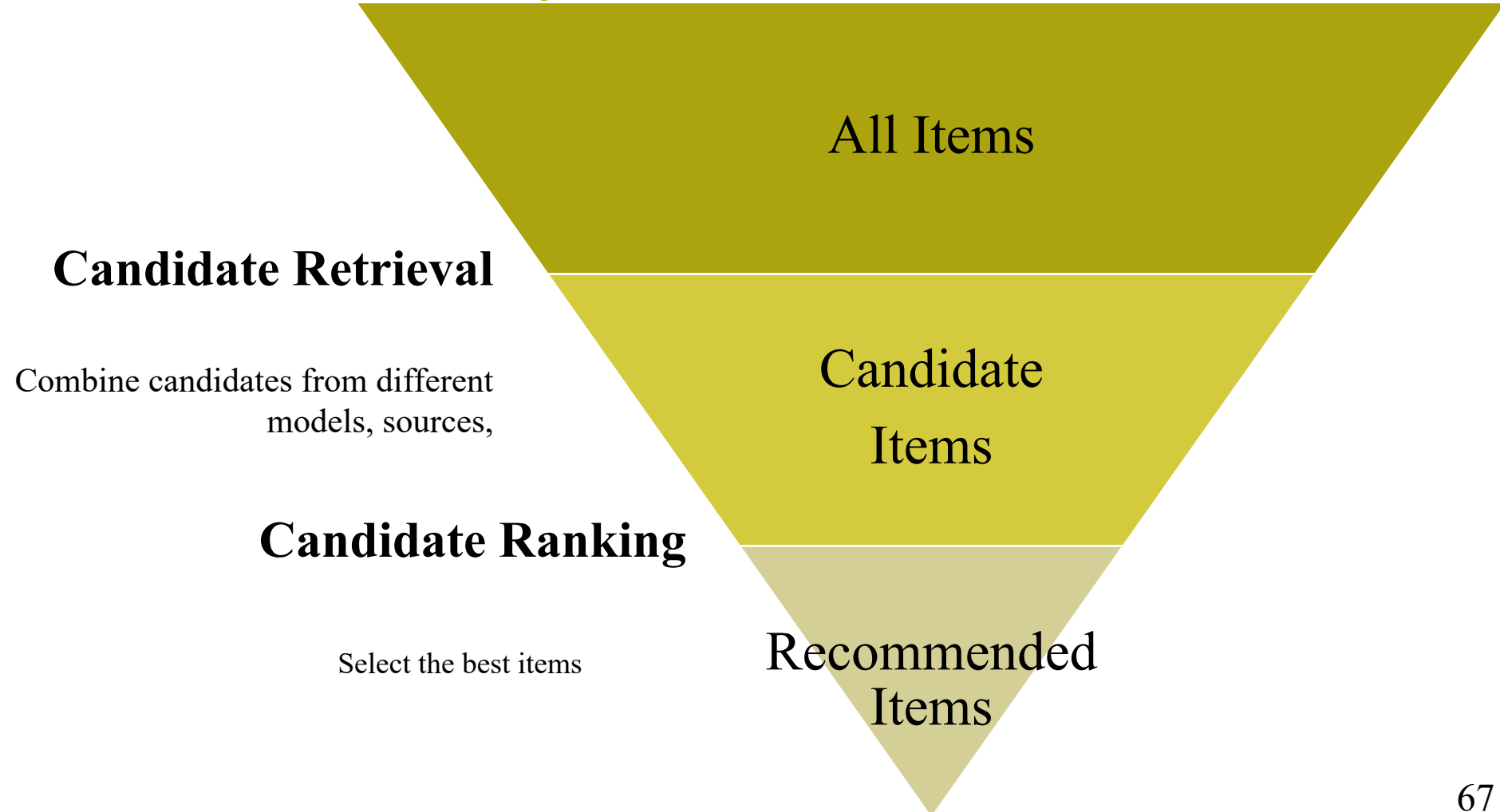
Switching/Ranking between recommendations from different models

- Select recommendations from a number of different models
- Which model/item to pick next is a multi-armed bandit problem or a ranking problem

Hybrid Algorithms

- Factorization Machines: Factorize the linear regression coefficients of user and item cross features
- Content-Boosted Collaborative Filtering: Use content-based model to complete the sparse ratings matrix and then apply (user-based) nearest neighborhood

Large Scale Recommender System Architecture





UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Wrap Up

Recommenders in Python

[Surprise](#) is a Python package implementing collaborative filtering-based recommenders.

```
: import surprise

: # The columns must correspond to user id, item id and ratings (in that order).
data = surprise.Dataset.load_from_df(df_ratings[['user_id', 'book_id', 'rating']], surprise.Reader(rating_scale=(1, 5)))

: # Split the data into a train and test set (20%) randomly
train,test = surprise.model_selection.split.train_test_split(data, test_size=0.2)

: # Train an Item-based Nearest neighbourhood model with 20 neighbours and using Cosine similarity as similarity measure
similarity = {"name": "cosine", "user_based": False} # Item Based Nearest Neighbourhood with Cosine similarity
algo = surprise.KNNBasic(k=20, sim_options=similarity)
algo.fit(train)

: # Make predictions for the ratings in the test set and get the accuracy of these predictions (rmse, precision at 10, recall at 10):
predictions = algo.test(test)
rmse = surprise.accuracy.rmse(predictions)
precision, recall = precision_recall_at_k(predictions, k=10, threshold=3.5)

: # get a prediction of the rating for user_id 1 on book_id 2
predicted_rating = algo.predict(uid=1, iid=2)
predicted_rating

: print(f'Predicted rating for user 1 on book 2: {predicted_rating.est}')
```



Today's Course Material

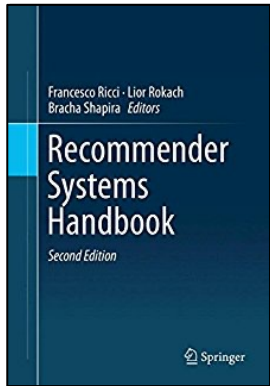
- Lecture notes
- **Computer Assignment**

Computer Assignment Guidelines

- Analysis and modeling on a dataset
- Submit a (concise) report and notebook (e.g., Google Colab link)

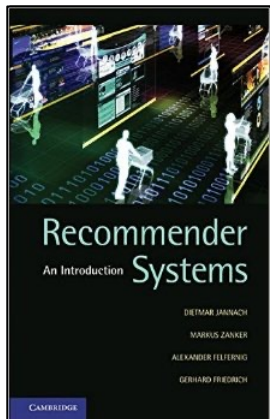
- Report: Do not only write what you did, but also
 - ☐ **Why** you did it: Motivate your assumptions and choices
 - ☐ What are the **results**: include charts/tables of the most important results in the report
 - ☐ **Interpretation** of the results: How do you interpret the results? What conclusions do you have and what's the impact for next steps?
- Code: Do not copy (major) code snippets from online sources
 - ☐ **Always cite all the resources you reference**
 - ☐ In case of exceptions/doubt: reference and motivate

If you want to know more...



Recommender Systems Handbook

- Ricci, Rokach, Shapira & Kantor (editors)



Recommender Systems: An Introduction

- Jannach, Zanker, Felfering & Friedrich